

# **Apprentissage Automatique et réseaux de neurones**

## ***Lecture 3: Réseaux de neurones***

***Mona Taghavi***

# Revision: Limites du Perceptron

- En 1969, Minsky et Papert ont montré formellement quelles fonctions pouvaient ou non être représentées par des perceptrons
- Seules les fonctions linéairement séparables peuvent être représentées par un perceptron

## Dartmouth Conference: The Founding Fathers of AI



John McCarthy



Marvin Minsky



Claude Shannon



Ray Solomonoff



Alan Newell

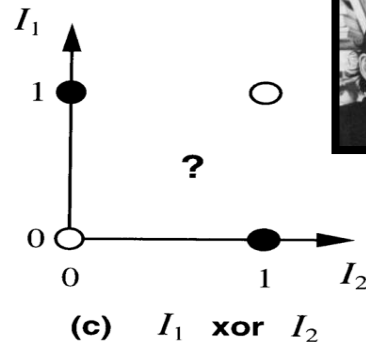
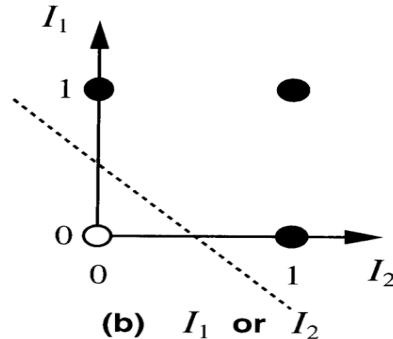
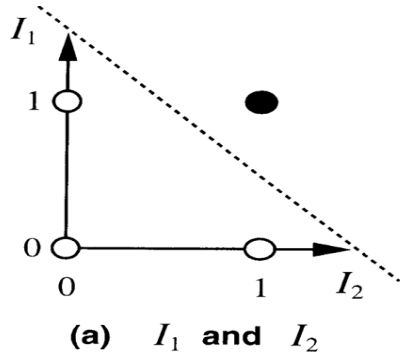


Herbert Simon

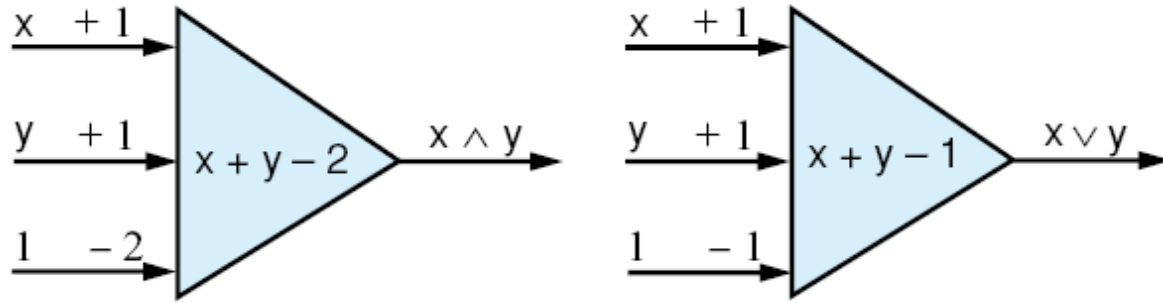


Arthur Samuel

And three others...  
Oliver Selfridge  
(Pandemonium theory)  
Nathaniel Rochester  
(IBM, designed 701)  
Trenchard More  
(Natural Deduction)



# Revision: Perceptrons AND et OR



| x | y | $x + y - 2$ | Output |
|---|---|-------------|--------|
| 1 | 1 | 0           | 1      |
| 1 | 0 | -1          | -1     |
| 0 | 1 | -1          | -1     |
| 0 | 0 | -2          | -1     |

# Revision: la fonction XOR

- On ne peut pas construire un perceptron pour apprendre la fonction OU-exclusif

- Pour apprendre XOR, avec :

- deux entrées  $x_1$  et  $x_2$
- deux poids  $w_1$  et  $w_2$
- un seuil  $\dagger$

| $x_1$ | $x_2$ | Sortie |
|-------|-------|--------|
| 1     | 1     | 0      |
| 1     | 0     | 1      |
| 0     | 1     | 1      |
| 0     | 0     | 0      |

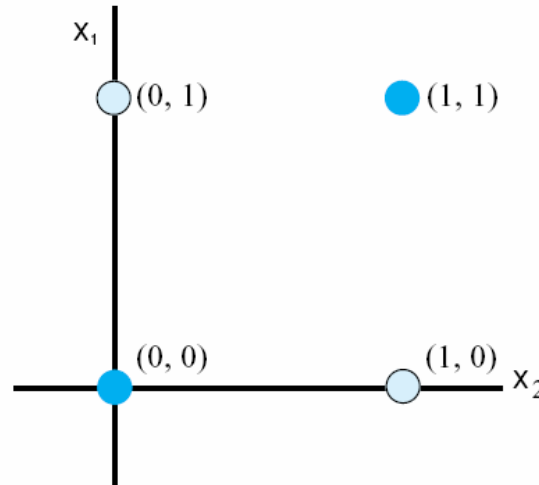
- c'est-à-dire, doit avoir :

- $(1 \times w_1) + (1 \times w_2) < \dagger$  (pour la 1ère ligne)
- $(1 \times w_1) + 0 \geq \dagger$
- $0 + (1 \times w_2) \geq \dagger$
- $0 + 0 < \dagger$

- **Ce qui n'a pas de solution... donc un perceptron ne peut pas apprendre XOR**

# Revision: La fonction XOR - Visuellement

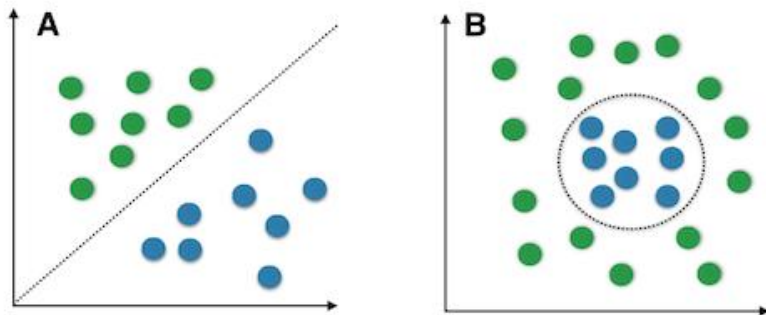
- Dans un espace bidimensionnel (2 caractéristiques pour  $X$ )
- Aucune ligne droite en deux dimensions ne peut séparer
  - $(0, 1)$  et  $(1, 0)$  de
  - $(0, 0)$  et  $(1, 1)$ .



# Revision: Fonctions non linéairement séparables

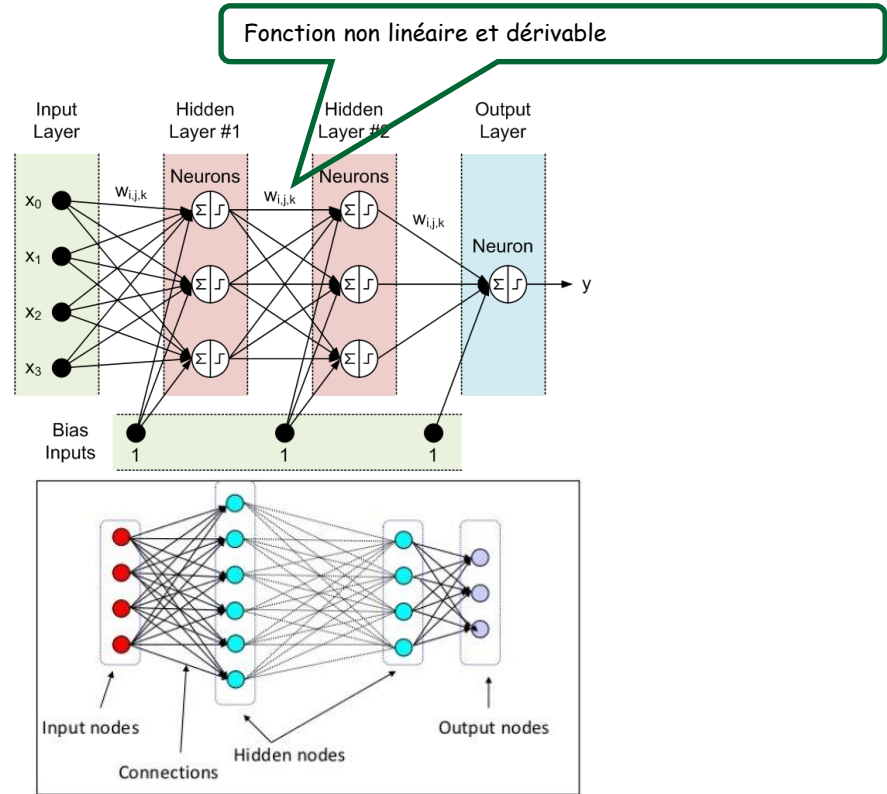
- Les problèmes du monde réel ne peuvent pas toujours être représentés par des fonctions linéairement séparables...
- Cela a entraîné une baisse de l'intérêt pour les réseaux de neurones dans les années 1970

Linear vs. nonlinear problems



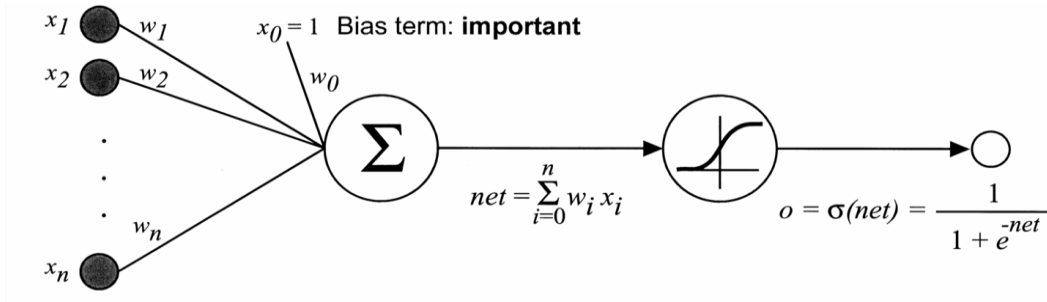
# Réseaux de neurones multicouches

- **Solution :**
  1. pour apprendre des fonctions plus complexes (frontières de décision plus complexes), avoir des nœuds cachés
  2. et pour les décisions non binaires, avoir plusieurs nœuds de sortie utiliser une fonction d'activation non linéaire



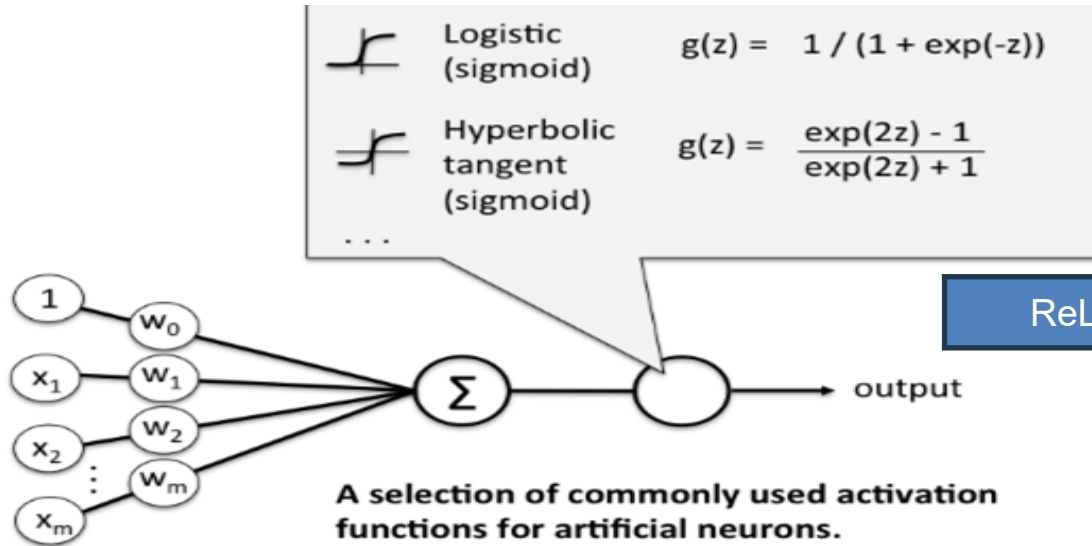
# La fonction sigmoïde

- La rétropropagation nécessite une fonction d'activation dérivable
- fonction sigmoïdale
- $f$  renvoie une valeur *entre* 0 et 1 (au lieu de 0 ou 1)
- $f$  indique à quel point la sortie du réseau est proche ou éloignée de la bonne réponse (le *terme d'erreur*)





# Fonctions d'activation typiques



ReLU est la plus récente.

# Apprentissage dans un réseau de neurones

- L'apprentissage est le même que dans un perceptron :
  - alimenter le réseau avec des données d'entraînement
  - s'il y a une erreur (une différence entre la sortie et la cible), ajuster les poids
- Nous devons donc évaluer la responsabilité d'une erreur par rapport aux poids qui y contribuent

# Feed-forward + Backpropagation

- Propagation avant :

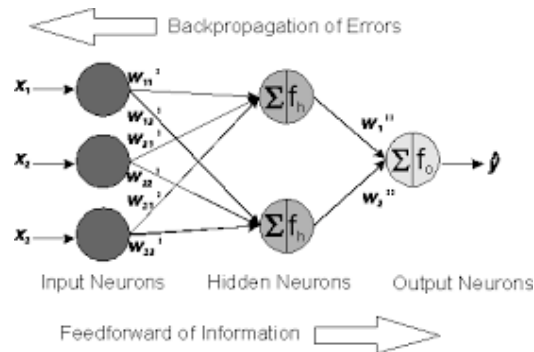
- L'entrée des caractéristiques est transmise dans le réseau de la couche d'entrée vers la couche de sortie

- Rétropropagation :

- le flux d'erreur revient de la couche de sortie vers la couche d'entrée (pour ajuster les poids afin de minimiser l'erreur de sortie)

- Répéter jusqu'à ce que le taux d'erreur soit minimisé

- répéter le passage avant et arrière pour les points de données suivants jusqu'à ce que tous les points de données soient examinés (1 époque)
- répéter tout cet exercice (plusieurs époques) jusqu'à ce que l'erreur globale soit minimisée



**Cost fonction** / **Loss function**

$$\text{mean squared errors} = \frac{1}{2} \frac{\sum_{i=1}^n (T_i - O_i)^2}{n} < \varepsilon$$

where  $\varepsilon \sim 0.0001$

n = nombre d'exemples d'entraînement

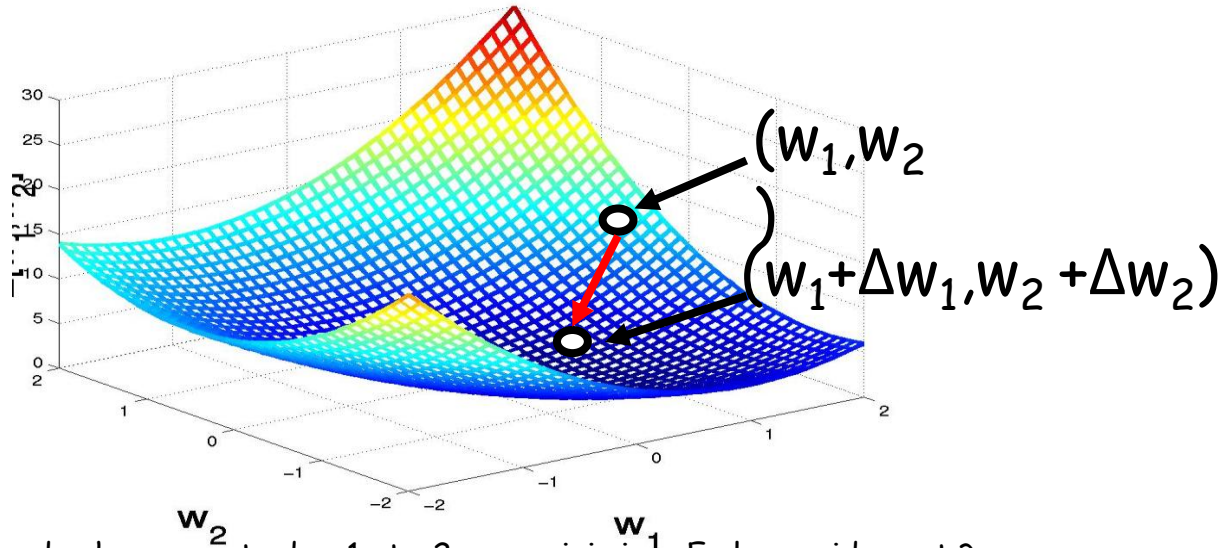
# Rétropropagation

- Dans un réseau multicouche...
  - Le calcul de l'erreur dans la couche de **sortie** est clair.
  - Le calcul de l'erreur dans la couche **cachée** n'est pas clair, car nous ne savons pas quelle devrait être sa sortie
- Intuitivement :
  - Un nœud caché  $h$  est « responsable » d'une fraction de l'erreur dans chacun des nœuds de sortie auxquels il est connecté.
- Ainsi, les valeurs d'erreur ( $\delta$ ) :
  - sont divisées en fonction du poids de leur connexion entre le nœud caché et le nœud de sortie
  - et sont propagées vers l'arrière pour fournir les valeurs d'erreur ( $\delta$ ) pour la couche cachée.

# Descente de gradient visuelle

$$E(w_1, w_2) = \frac{1}{2} \frac{\sum_{i=1}^n (T_i - O_i)^2}{n}$$

- Objectif : minimiser  $E(w_1, w_2)$  en modifiant  $w_1$  et  $w_2$

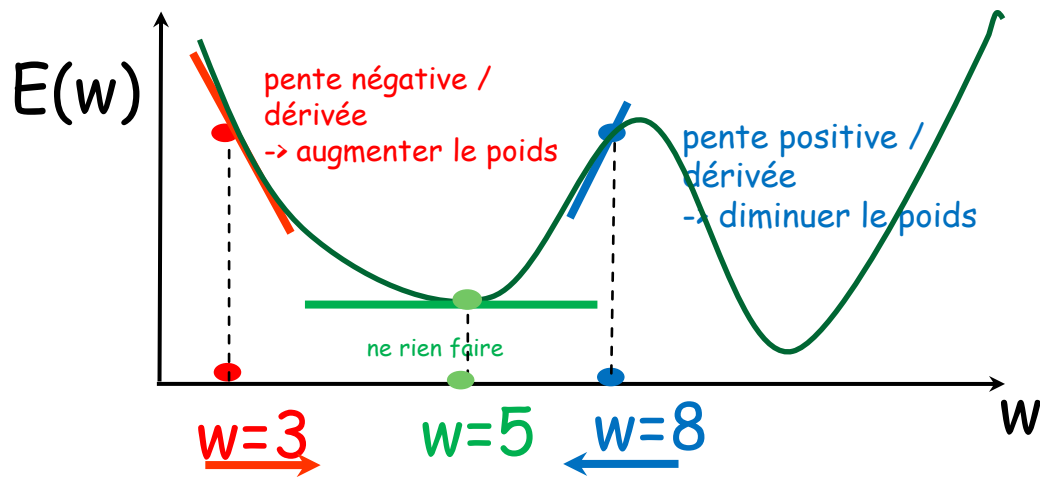


- Mais quelle est la meilleure combinaison de changements de  $w_1$  et  $w_2$  pour minimiser  $E$  plus rapidement ?
- La règle delta est une technique de descente de gradient pour mettre à jour les poids dans un perceptron monocouche.
- **Le gradient**  $\nabla E$  pointe vers la direction opposée à la descente la plus raide de  $E(w_1, w_2)$ , c'est-à-dire une approche de type « hill-climbing » (ascension de colline)...

# Gradients

Le gradient n'est qu'une dérivée en 1D

Ex: la dérivée est :  $E(w) = (w - 5)^2$



$$\frac{\partial}{\partial w} E = 2(w - 5)$$

Si  $w=3$   $\frac{\partial}{\partial w} E(3) = 2(3 - 5) = -4$

la dérivée dit d'augmenter  $w$   
(aller dans la direction opposée  
de la dérivée)

Si  $w=8$   $\frac{\partial}{\partial w} E(8) = 2(8 - 5) = 6$

la dérivée dit de diminuer  $w$   
(aller dans la direction opposée  
de la dérivée)

# Entraînement du réseau

- Étape 0 : Initialiser les poids du réseau de manière aléatoire

// propagation avant (feedforward)

- Étape 1 : Effectuer un passage avant (forward pass) à travers le réseau (utiliser la sigmoïde)

$$O_i = g\left(\sum_j w_{ji} x_j\right) = \text{sigmoid}\left(\sum_j w_{ji} x_j\right) = \frac{1}{1 + e^{-\left(\sum_j w_{ji} x_j\right)}}$$

Dérivée de la sigmoïde

// propager les erreurs vers l'arrière (backwards)

- Étape 2 : Pour chaque unité de **sortie**  $k$ , calculer son terme d'erreur  $\delta_k$

$$\delta_k \leftarrow g'(x_k) \times \text{Err}_k = O_k (1 - O_k) \times (O_k - T_k)$$

note, if we use  $g = \text{sigmoid}$  :

$$g'(x) = g(x) (1 - g(x))$$

- Étape 3 : Pour chaque unité **cachée**  $h$ , calculer son terme d'erreur  $\delta_h$

Somme du terme d'erreur pondéré des nœuds de sortie auxquels  $h$  est connecté (c'est-à-dire que  $h$  a contribué aux erreurs  $\delta_k$ )

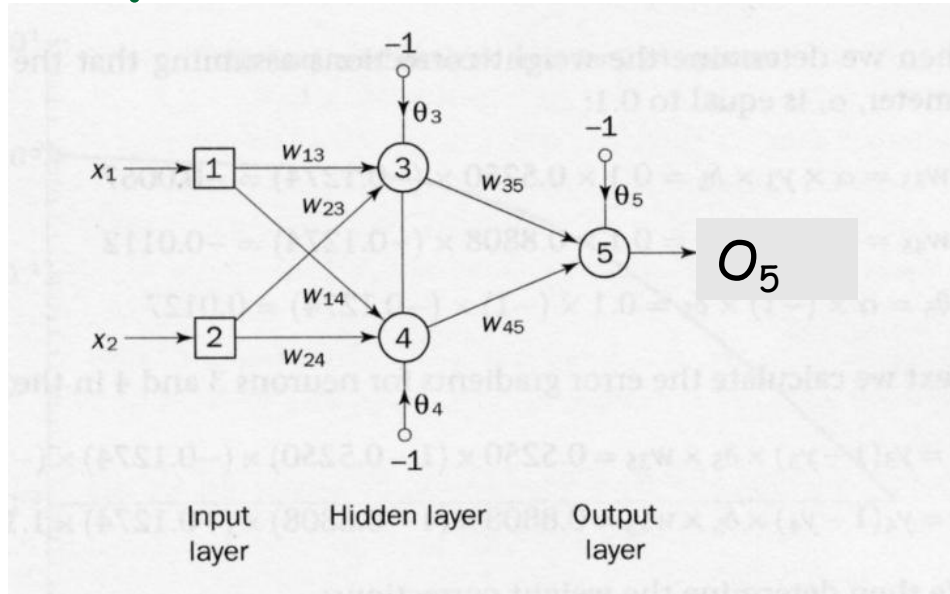
- Étape 4 : Mettre à jour chaque poids du réseau  $w_{ij}$  :

$$\delta_h \leftarrow g'(x_h) \times \text{Err}_h = O_h (1 - O_h) \times \sum_{k \in \text{outputs}} w_{hk} \delta_k$$

- Répéter les étapes 1 à 4 jusqu'à ce que l'erreur soit minimisée à un niveau donné

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij} \text{ where } \Delta w_{ij} = -\eta \delta_j O_i$$

# Exemple : XOR

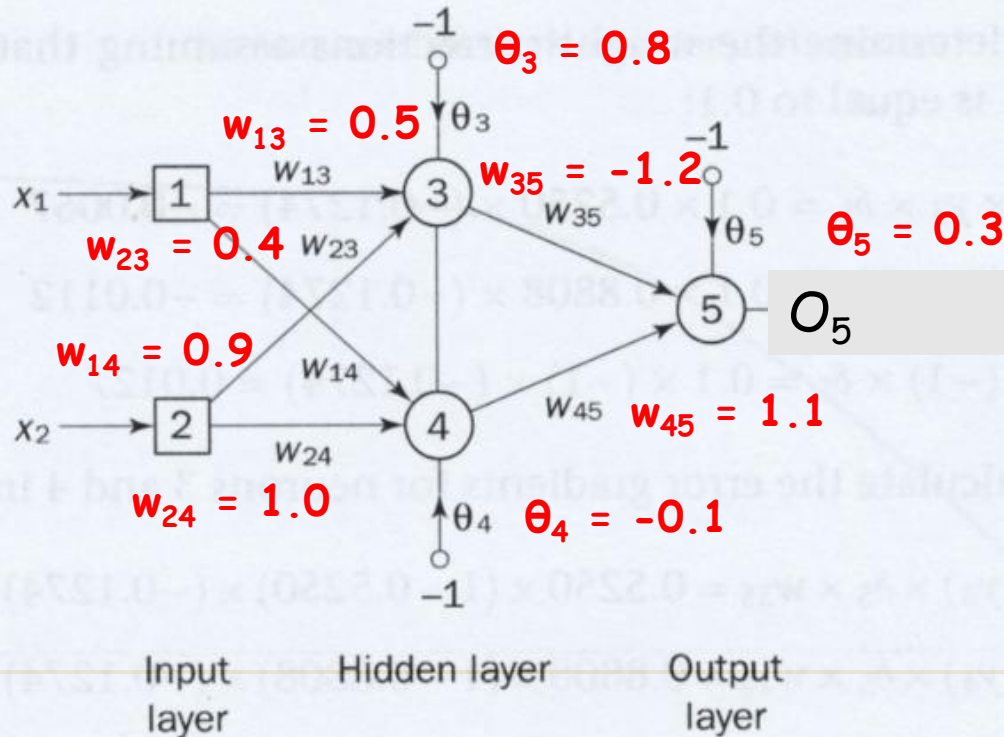


- 2 entrées + 2 neurones cachés + 1 neurone de sortie + 3 biais (-1)



# Exemple : Étape 0 (initialisation)

- Étape 0 : Initialiser le réseau de manière aléatoire



# Étape 1 : Feed Forward

Étape 1 : Fournir les entrées et calculer la sortie

$$O_i = \text{sigmoid}\left(\sum_j w_{ji} x_j\right) = \frac{1}{1 + e^{-\left(\sum_j w_{ji} x_j\right)}}$$

| $x_1$ | $x_2$ | Target output T |
|-------|-------|-----------------|
| 1     | 1     | 0               |
| 0     | 0     | 0               |
| 1     | 0     | 1               |
| 0     | 1     | 1               |

□ Sortie du nœud caché 3 :

$(x_1=1, x_2=1)$

■ Sortie du nœud caché 3 :

□  $O_3 = \text{sigmoid}(x_1 w_{13} + x_2 w_{23} - \theta_3) = 1 / (1 + e^{-(1 \times 0.5 + 1 \times 0.4 - 1 \times 0.8)}) = 0.5250$

■ Sortie du nœud caché 4 :

□  $O_4 = \text{sigmoid}(x_1 w_{14} + x_2 w_{24} - \theta_4) = 1 / (1 + e^{-(1 \times 0.9 + 1 \times 1.0 + 1 \times 0.1)}) = 0.8808$

■ Sortie du nœud caché 5 :

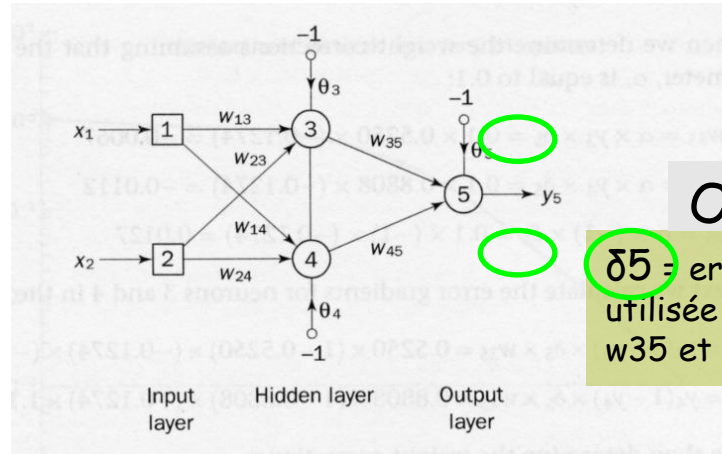
□  $O_5 = \text{sigmoid}(O_3 w_{35} + O_4 w_{45} - \theta_5) = 1 / (1 + e^{-(0.5250 \times 1.2 + 0.8808 \times 1.1 - 1 \times 0.3)}) = 0.5097$

## Étape 2 : Calculer le terme d'erreur de la couche de sortie

$$\delta_k \leftarrow g'(x_k) \times \text{Err}_k = O_k (1 - O_k) \times (O_k - T_k)$$

- Terme d'erreur du neurone 5 dans la couche de sortie :

- $\delta_5 = O_5 (1 - O_5) (O_5 - T_5)$   
 $= (0.5097) \times (1 - 0.5097) \times (0.5097 - 0)$   
 $= 0.1274$



$O_5$

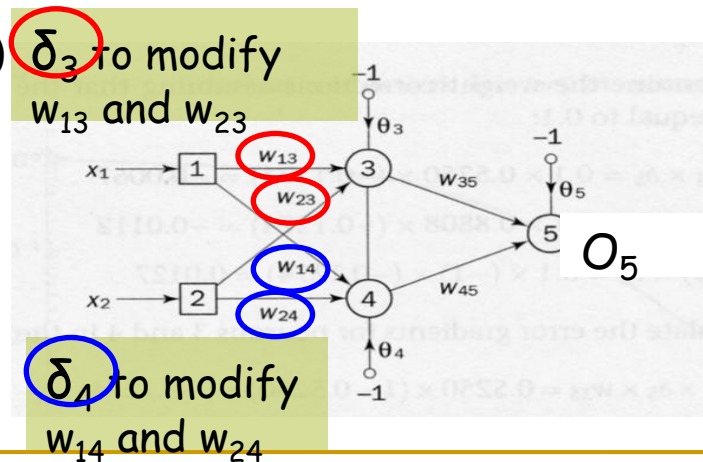
$\delta_5$  = erreur... sera utilisée pour modifier  $w_{35}$  et  $w_{45}$

## Étape 3 : Calculer le terme d'erreur de la couche cachée

$$\delta_h \leftarrow g'(x_h) \times \text{Err}_h = O_h(1 - O_h) \times \sum_{k \in \text{outputs}} \delta_k w_{hk}$$

Terme d'erreur des neurones 3 et 4 :

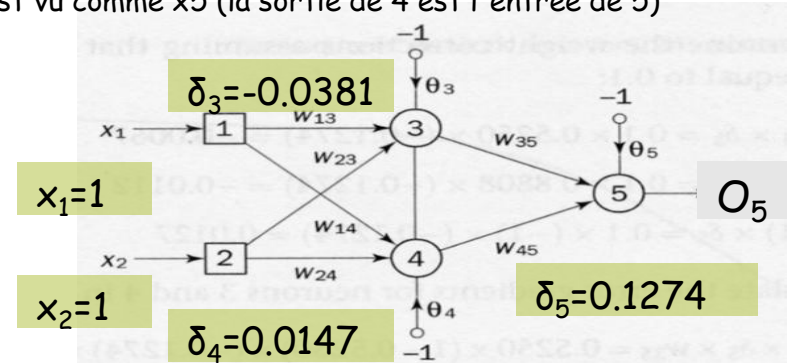
- $\delta_3 = O_3 (1 - O_3) \delta_5 w_{35}$   
 $= (0.5250) \times (1 - 0.5250) \times (0.1274) \times (-1.2)$   
 $= -0.0381$
- $\delta_4 = O_4 (1 - O_4) \delta_5 w_{45}$   
 $= (0.8808) \times (1 - 0.8808) \times (0.1274) \times (1.1)$   
 $= 0.0147$



# Étape 4 : Changements de poids

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij} \text{ where } \Delta w_{ij} = -\eta \delta_j x_i$$

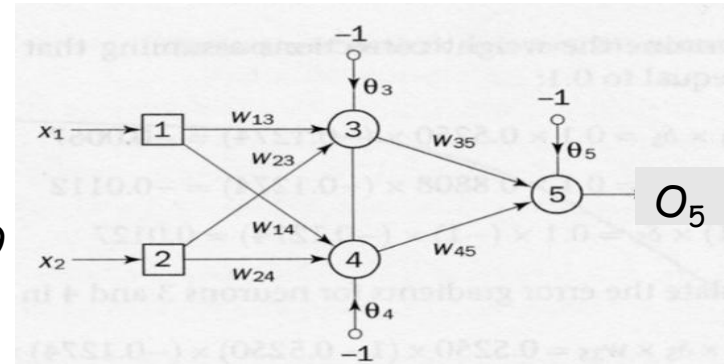
- Mettre à jour tous les poids (en supposant un taux d'apprentissage constant  $\eta = 0,1$ )
- $\Delta w_{13} = -\eta \delta_3 x_1 = -0,1 \times -0,0381 \times 1 = 0,0038$
- $\Delta w_{14} = -\eta \delta_4 x_1 = -0,1 \times 0,0147 \times 1 = -0,0015$
- $\Delta w_{23} = -\eta \delta_3 x_2 = -0,1 \times -0,0381 \times 1 = 0,0038$
- $\Delta w_{24} = -\eta \delta_4 x_2 = -0,1 \times 0,0147 \times 1 = -0,0015$
- $\Delta w_{35} = -\eta \delta_5 O_3 = -0,1 \times 0,1274 \times 0,5250 = -0,00669$  // O3 est vu comme x5 (la sortie de 3 est l'entrée de 5)
- $\Delta w_{45} = -\eta \delta_5 O_4 = -0,1 \times 0,1274 \times 0,8808 = -0,01122$  // O4 est vu comme x5 (la sortie de 4 est l'entrée de 5)
- $\Delta \theta_3 = -\eta \delta_3 (-1) = -0,1 \times -0,0381 \times -1 = -0,0038$
- $\Delta \theta_4 = -\eta \delta_4 (-1) = -0,1 \times 0,0147 \times -1 = 0,0015$
- $\Delta \theta_5 = -\eta \delta_5 (-1) = -0,1 \times 0,1274 \times -1 = 0,0127$



# Étape 5 : Mettre à jour les poids

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij} \text{ where } \Delta w_{ij} = -\eta \delta_j x_i$$

- Mettre à jour tous les poids (en supposant un taux d'apprentissage constant  $\eta = 0.1$ )
  - $w_{13} = w_{13} + \Delta w_{13} = 0.5 + 0.0038 = 0.5038$
  - $w_{14} = w_{14} + \Delta w_{14} = 0.9 - 0.0015 = 0.8985$
  - $w_{23} = w_{23} + \Delta w_{23} = 0.4 + 0.0038 = 0.4038$
  - $w_{24} = w_{24} + \Delta w_{24} = 1.0 - 0.0015 = 0.9985$
  - $w_{35} = w_{35} + \Delta w_{35} = -1.2 - 0.00669 = -1.20669$
  - $w_{45} = w_{45} + \Delta w_{45} = 1.1 - 0.01122 = 1.08878$
  - $\theta_3 = \theta_3 + \Delta \theta_3 = 0.8 - 0.0038 = 0.7962$
  - $\theta_4 = \theta_4 + \Delta \theta_4 = -0.1 + 0.0015 = -0.0985$
  - $\theta_5 = \theta_5 + \Delta \theta_5 = 0.3 + 0.0127 = 0.3127$



# Étape 6 : Itérer à travers les données

- après avoir ajusté tous les poids, répétez la passe avant et la passe arrière pour le point de donnée suivant jusqu'à ce que tous les points de données soient examinés
- répétez tout cet exercice jusqu'à ce que l'erreur globale soit minimisée

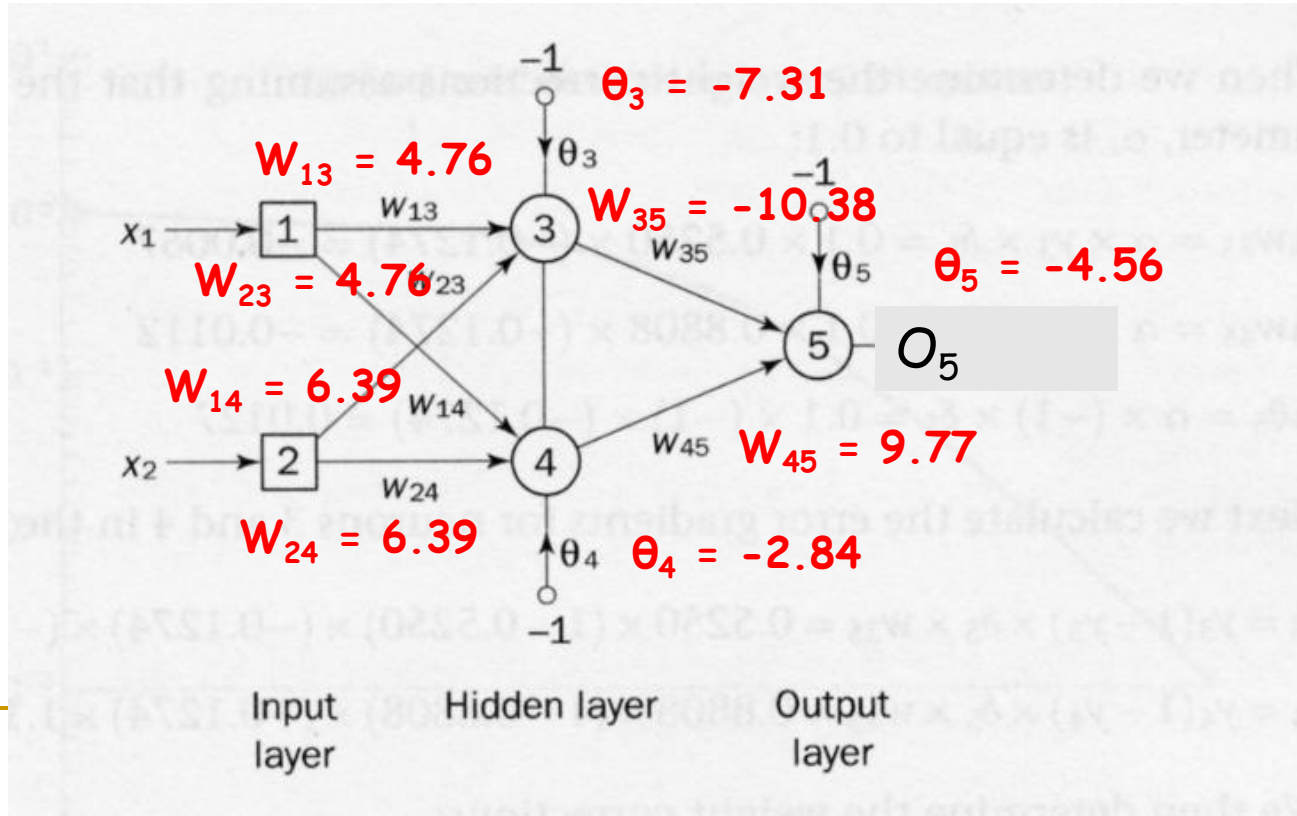
○ Ex : Erreur quadratique moyenne (MSE) =

$$\frac{1}{2} \frac{\sum_{i=1}^n (T_i - O_i)^2}{n} < \varepsilon$$

où  $\varepsilon \sim 0.0001$  et  $n$  = nb d'exemples d'entraînement

# Le résultat...

- Après 224 époques, nous obtenons :
  - (1 époque = parcourir toutes les données une fois)





# L'erreur est minimisée

| Inputs         |                | Target Output<br>T | Actual Output<br>O | Error<br>T-O |
|----------------|----------------|--------------------|--------------------|--------------|
| x <sub>1</sub> | x <sub>2</sub> |                    |                    |              |
| 1              | 1              | 0                  | 0.0155             | -0.0155      |
| 0              | 1              | 1                  | 0.9849             | 0.0151       |
| 1              | 0              | 1                  | 0.9849             | 0.0151       |
| 0              | 0              | 0                  | 0.0175             | -0.0175      |

$$\begin{aligned}\text{Mean Squared Error} &= \frac{1}{2} \cdot \frac{(-0.0155^2 + 0.0151^2 + 0.0151^2 + 0.0175^2)}{4} \\ &= 0.000125 < 0.001 \text{ (some threshold } \varepsilon) \dots \text{ stop!}\end{aligned}$$



local minimum...

# Descente de Gradient Stochastique

- Descente de Gradient en Rafale (Batch GD)
  - met à jour les poids après 1 époque
  - peut être coûteux (temps et mémoire) car nous devons évaluer l'ensemble du jeu de données d'entraînement avant de faire un pas vers le minimum.
- Descente de Gradient Stochastique (SGD)
  - met à jour les poids après chaque exemple d'entraînement
  - converge souvent plus rapidement par rapport au GD
  - mais la fonction d'erreur n'est pas aussi bien minimisée que dans le cas du GD
  - pour obtenir de meilleurs résultats, mélangez le jeu d'entraînement pour chaque époque
- Descente de Gradient par Mini-Rafale :
  - compromis entre GD et SGD
  - divisez votre jeu de données en sections, et mettez à jour les poids après l'entraînement sur chaque section

# Réseaux de Neurones

- Inconvénient :
  - le résultat n'est pas facile à comprendre pour les humains (ensemble de poids comparé à un arbre de décision)... c'est une boîte noire
- Avantage :
  - robuste au bruit dans l'entrée (de petits changements dans l'entrée ne provoquent normalement pas de changement dans la sortie) et dégradation progressive